

Sri Lanka Institute of Information Technology



IT3021 : Data Warehousing and Business Intelligence

Year 3 Semester 2 – 2026

Assignment -1

Withana N.K. – IT23189676

Table of Contents

1.Introduction.....	3
2.Data set selection	4
2.1 Introduction.....	4
2.2 Dataset Overview.....	4
2.3 Justification as an OLTP Dataset	5
2.4 Time Coverage of Dataset.....	5
2.5 Suitability for Data Warehousing and BI.....	5
2.6 ER-diagram	6
3.Preparation of data sources	8
3.1 Data Source Integration	10
3.2 Data Enrichment and Transformation.....	10
3.3 Suitability for ETL and Data Warehousing.....	10
4.Solution Architecture	11
5.Data warehouse design & development.....	15
6.ETL Development.....	23
7.ETL development – Accumulating fact tables.....	36
8.Conclusion	40

1.Introduction

In today's data-driven business environment, e-commerce platforms generate large volumes of operational data that can be transformed into actionable insights through data warehousing and business intelligence. Online retail systems continuously capture customer records, order transactions, product details, seller information, delivery timelines, and pricing attributes. However, this data usually originates from different source formats and operational structures, making direct analytical reporting difficult without a dedicated integration and transformation process.

This assignment focuses on the design and implementation of a complete data warehouse solution for an **ecommerce order processing environment** based on the Olist marketplace dataset. The selected dataset represents a realistic online retail scenario containing customers, orders, order items, sellers, and products. The source data is distributed across heterogeneous formats including CSV, and SQL script files, which makes it suitable for demonstrating practical data integration, staging, ETL, dimensional modeling, and analytical processing.

The implementation was carried out in several stages: dataset selection and validation, preparation of multiple source types, construction of a high-level DW & BI architecture, design of a dimensional model using a star schema, implementation of the schema in SQL Server, development of ETL packages using SQL Server Integration Services (SSIS), and extension of the fact table with accumulating snapshot attributes. The resulting solution demonstrates how raw operational data can be transformed into a clean, integrated, query-ready warehouse that supports future reporting, OLAP analysis, and dashboarding.

2.Data set selection

2.1 Introduction

For this assignment, a real-world e-commerce dataset based on the Olist marketplace was selected as the OLTP source dataset. The dataset represents the core operational processes of an online retail platform and includes customer details, order transactions, order item lines, seller information, and product attributes. It captures the life cycle of online retail activity from customer registration and order placement to product level transaction lines fulfilled by sellers.

The selected dataset is not a pre-built analytical dataset and does not contain any existing fact table or dimension table structures. Therefore, it satisfies the requirement of being a genuine OLTP-style operational dataset suitable for data warehouse design and ETL implementation. In addition, it is not based on Adventure Works and was not taken from practical class examples, making it an appropriate and sufficiently novel choice for this assignment.

The business scenario represented by the dataset is highly suitable for illustrating core data warehousing concepts such as data integration, staging, ETL processing, surrogate key generation, slowly changing dimensions, dimensional modeling, accumulating facts, and analytical reporting.

2.2 Dataset Overview

The original dataset is composed of multiple source files: olist_customers.csv, olist_orders.csv, olist_order_items.csv, olist_sellers.csv, and olist_products.sql. Together these files describe the operational data of an e-commerce marketplace. The dataset includes 99,441 customer records, 99,441 orders, 112,650 order item rows, 3,095 sellers, and approximately 32,919 products in the cleaned SQL script.

Based on the order purchase timestamp, the data covers a period from 2016-09-04 to 2018-10-17, spanning approximately 772 days. This provides more than one year of time-based transactional coverage and therefore supports the creation of time hierarchies such as Year > Quarter > Month > Day in the warehouse.

Data source - <https://www.kaggle.com/datasets/burayamail/brazilian-e-commerce>

Key Entities:

- **Customer:** Contains customer master data including customer identifiers, city, state, and zip code prefix
- **Order:** Stores order-level information such as order status and purchase, approval, carrier customer delivery, and estimated delivery timestamps.
- **Order Item:** Represents the transaction line grain of the dataset and includes product, seller, price, freight value, and shipping limit details.
- **Seller:** Contains seller location data including city, state, and zip code prefix.
- **Product:** Contains product master data such as product category and descriptive/physical measurements.

2.3 Justification as an OLTP Dataset

The selected Olist dataset is an operational transactional dataset because it stores raw business events rather than pre-aggregated analytics. Each order and order item row corresponds to a business transaction, while customer, seller, and product tables store supporting operational master data.

The dataset is not organized into a star schema and does not include pre-defined facts or dimensions. Instead, the entities are linked through natural business keys such as `customer_id`, `order_id`, `product_id`, and `seller_id`. This confirms that the dataset is OLTP-oriented and therefore appropriate for a full warehouse design exercise.

2.4 Time Coverage of Dataset

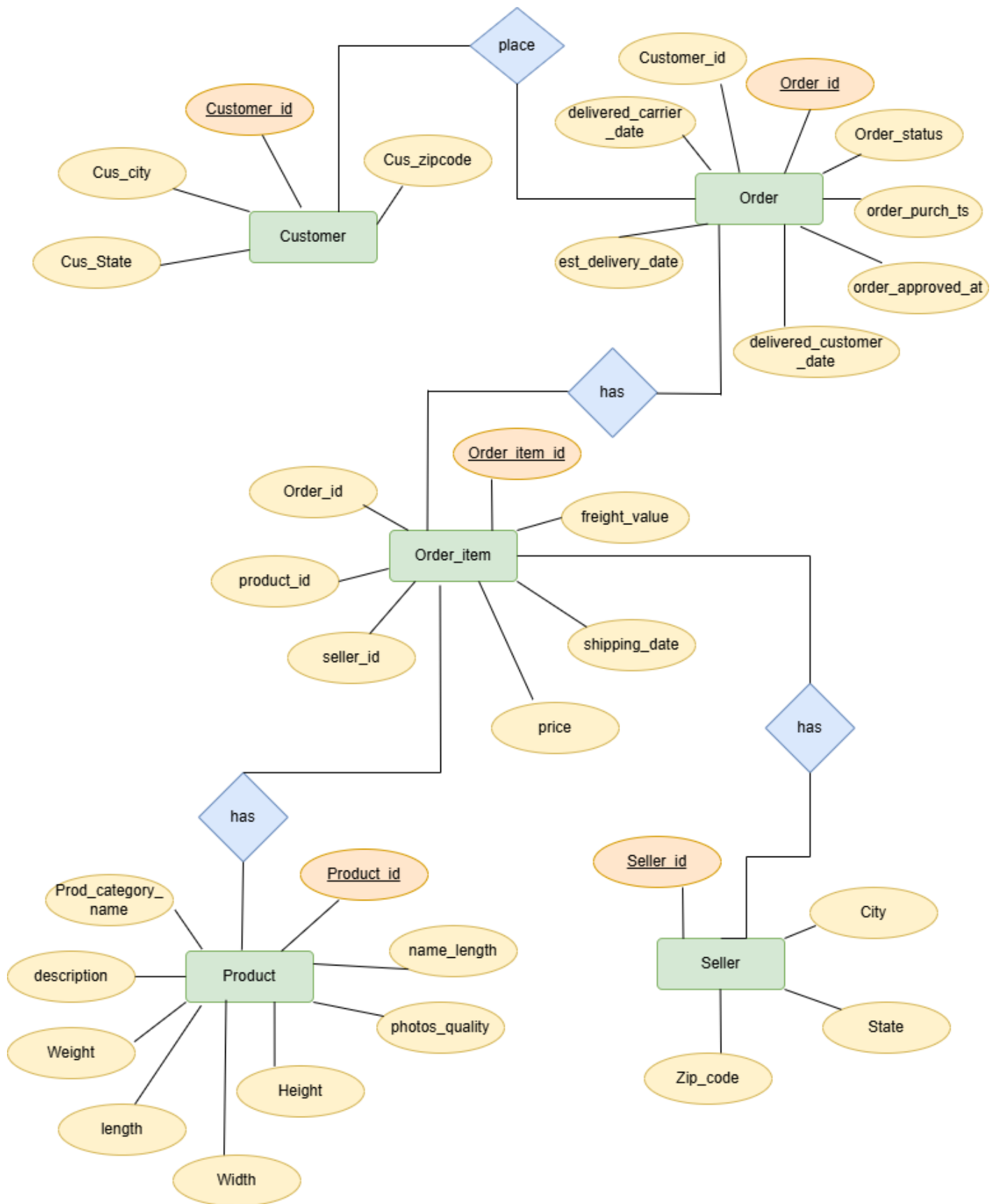
The dataset includes the `order_purchase_timestamp` field, which allows retail activities to be analyzed over time. The dataset was extended to include about two year of data, to have enough information based on time for analysis. This allows the establishment of time-based hierarchies like Year, Month and Day in the data warehouse.

2.5 Suitability for Data Warehousing and BI

The chosen dataset is highly suitable for data warehousing and business intelligence because it contains rich transactional relationships and multiple analytical perspectives. Order item data supports the creation of a central fact table, while customer, product, seller, and date data naturally support dimension tables.

The dataset also enables hierarchical analysis, such as date hierarchies and geographic hierarchies. Typical analytical questions supported by this dataset include sales by product category, sales by seller location, order trends over time, customer distribution by state, and freight cost analysis.

2.6 ER-diagram



The system represents an **E-commerce Order Processing System** used to manage how customers buy products online, how orders are handled, and how sellers fulfill those orders.

A **Customer** signs up on the platform and has a profile that includes basic details like their location (city, state, and zip code). Once registered, a customer can place multiple orders over time, but each order belongs to only one customer.

An **Order** is created whenever a customer makes a purchase. It keeps track of important information such as when the order was placed, whether it was approved, its current status, and delivery-related dates (like estimated and actual delivery times). Each order is linked to a single customer, but a customer can have many orders.

Every order is made up of one or more **Order Items**. These represent the individual products included in the order. For example, if a customer buys three different products, there will be three order items. Each order item includes details like the product, the seller, price, shipping date, and delivery cost. While an order can contain many items, each item belongs to just one order.

Each order item is connected to a **Product**, which contains details such as the product category, description, size, weight, and quality of images. A single product can appear in many different orders.

At the same time, each order item is fulfilled by a **Seller**. Sellers are responsible for providing and shipping the products. Their information includes location details like city, state, and zip code. A seller can handle many order items, but each order item is handled by only one seller.

Overall, this system shows how an online shopping platform works behind the scenes from customer registration and placing orders to product handling, seller involvement, and delivery tracking. It helps businesses manage orders efficiently while also making it easier to analyze customer behavior, sales trends, and delivery performance.

3.Preparation of data sources

The selected Olist E-commerce dataset was prepared by organizing the original data into multiple heterogeneous data sources. In real-world data warehousing environments, data is typically distributed across different systems and formats rather than being stored in a single unified file.

To simulate this real-world scenario, the dataset was divided into multiple source types including Excel files and SQL scripts. This approach enables the demonstration of data integration from different formats and supports the implementation of ETL processes using SQL Server Integration Services (SSIS).

1. Customer Data – CSV Source

- **Source:** olist_customers.csv
- **Type:** CSV (Comma-Separated Values)
- This source contains customer-related information, including:
 - ✓ customer_id
 - ✓ customer_unique_id
 - ✓ customer_zip_code_prefix
 - ✓ customer_city
 - ✓ customer_state
- This represents **master data**, which is relatively stable and is used to populate the **Customer Dimension (DimCustomer)** in the data warehouse.

2. Order Data – CSV Source

- **Source:** olist_orders.csv
- **Type:** CSV (Comma-Separated Values)
- This source contains the order-related data, including:
 - ✓ order_id
 - ✓ customer_id
 - ✓ order_status
 - ✓ order_purchase_timestamp
 - ✓ order_approved_at
 - ✓ order_delivered_carrier_date
 - ✓ order_delivered_customer_date
 - ✓ order_estimated_delivery_date
- This is the **main transactional dataset** and is used to populate: support the creation of the **fact table (fact_sales)** and date-related analysis.

3. Oredr Item Data – CSV Source

- **Source:** olist_order_items.csv
- **Type:** CSV (Comma-Separated Values)
- This source contains item -level transactional data such as:
 - ✓ order_id
 - ✓ order_item_id
 - ✓ product_id
 - ✓ seller_id
 - ✓ price
 - ✓ freight_value
- This data is used to build the **Fact table (fact_sales)**.

4. Seller Data – CSV Source

- **Source:** olist_sellers.csv
- **Type:** CSV (Comma-Separated Values)
- This source contains seller related information, such as:
 - ✓ seller_id
 - ✓ seller_zip_code_prefix
 - ✓ seller_city
 - ✓ seller_state
- This data is used to populate the **Seller Dimension (dim_seller)**.

5. Product Data – SQL Source

- **Source:** olist_products.sql
- **Type:** Relational Database Script (SQL Server)
- This source contains seller related information, such as:
 - ✓ product_id
 - ✓ product_category_name
 - ✓ product_name_length
 - ✓ product_description_length
 - ✓ product_photos_qty
 - ✓ product_weight_g
 - ✓ product_length_cm

✓ product_height_cm

✓ product_width_cm

- This data is used to populate the **Product Dimension (dim_product)**.

3.1 Data Source Integration

The separated data sources are integrated using common identifiers to maintain relationships between entities:

- **customer_id** → links customer data with orders
- **order_id** → connects orders and order items
- **product_id** → links order items with products
- **seller_id** → connects order_items with sellers

These keys ensure referential integrity and enable the integration of data from multiple sources during the ETL process.

3.2 Data Enrichment and Transformation

To enhance analytical capabilities and support data warehousing requirements, several transformations are planned and implemented during the ETL process.

- Creation of standard Date dimension based on order_purchase_timestamp
- Population of `total\_amount` using price and freight_value in the fact table
- Standardization of inconsistent or missing values (e.g., replacing '?' with 'N/A')
- Implementation of **Slowly Changing Dimensions (Type 2)** for customer data
- Generation of surrogate keys during ETL processing

These transformations are not part of the original source data but are applied during the ETL phase to improve data quality and ensure that the dataset is suitable for dimensional modeling and analytical queries.

3.3 Suitability for ETL and Data Warehousing

The prepared data sources effectively support data warehousing and ETL processes.

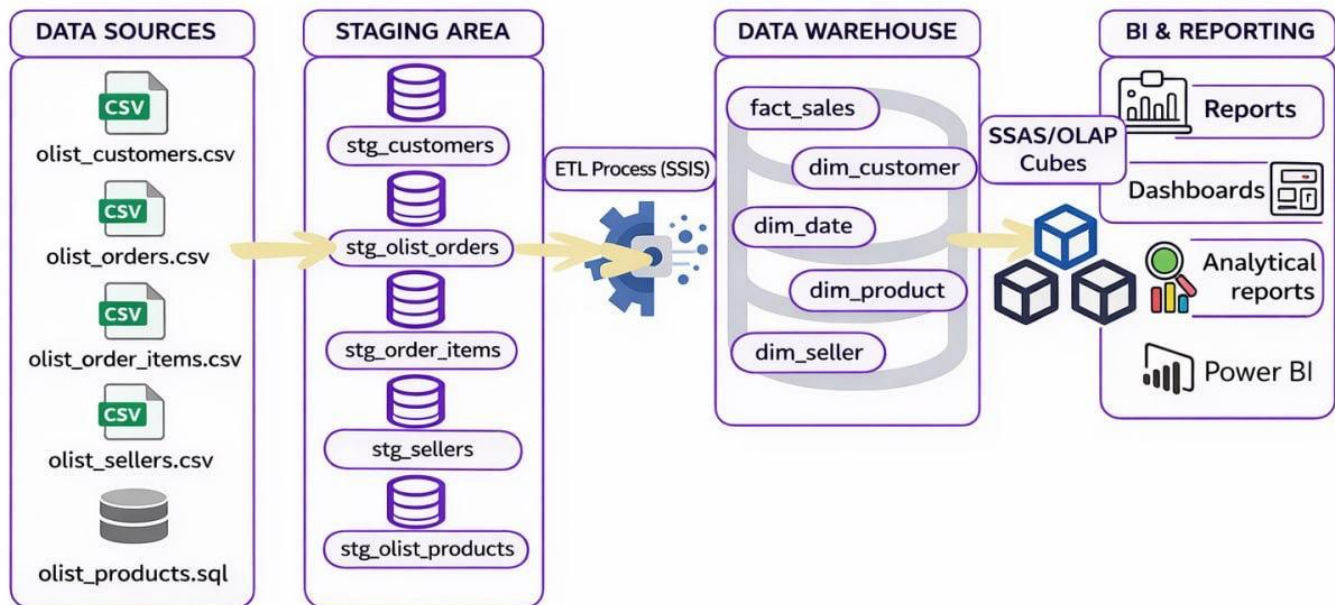
- ✓ Multiple heterogeneous data sources (SQL, CSV)
- ✓ Clear separation of transactional and dimensional data
- ✓ Availability of keys for data integration
- ✓ Rich set of attributes for transformation and analysis
- ✓ Support for building dimensions, fact tables, and hierarchies

This setup enables the implementation of ETL processes using tools such as SSIS and supports the development of OLAP cubes, dashboards, and analytical reports in later stages of the assignment.

4.Solution Architecture

A High-Level Data Warehouse and Business Intelligence (DW & BI) Solution Architecture has been designed to integrate data from multiple Heterogeneous sources and transform the data to meaningful analytical insights. The architecture is based on using a standard layered architecture comprised of data sources, staging area, ETL processes, data warehouse, OLAP cubes, and reporting tools.

This architecture provides efficient data integration, transformation, storage, and analysis, supporting business intelligence operations like reporting, dashboards, and advanced analytics.



1. Data Sources Layer

This layer contains the original heterogeneous data sources.

- ✓ **olist_customers.csv(CSV file)** → Customer data
- ✓ **olist_orders.csv (CSV file)** → order related data
- ✓ **olist_order_items.csv(CSV file)** → order item transaction data
- ✓ **olist_sellers.csv(CSV file)** → Seller data
- ✓ **olist_products.sql (SQL file)** → Product data

In the context of this **E-commerce Order Processing System**, this layer provides the raw transactional and master data that forms the foundation of the entire data warehouse solution. The heterogeneous sources customer data, order and order item transactions, seller information, and product details reflect how data is realistically distributed across different systems in an e-commerce environment. In the context of this **E-commerce Order Processing System**, this layer provides the raw transactional and master data that forms the foundation of the entire data warehouse solution. The heterogeneous sources customer data, order and order item transactions, seller information, and product details reflect how data is realistically distributed across different systems in an e-commerce environment.

2. Staging Area

The staging area is a temporary storage layer used to prepare data before loading into the data warehouse.

- ✓ **stg_customers**
 - ✓ **stg_orders**
 - ✓ **stg_order_items**
 - ✓ **stg_sellers**
 - ✓ **stg_products**
- Data extraction from source systems
 - Data cleaning and validation
 - Temporary storage before transformation

In the context of this **E-commerce Order Processing System**, the staging area acts as an intermediate layer where raw data extracted from multiple sources is temporarily stored, validated, and prepared before being loaded into the data warehouse. This layer is essential because the source data originates from heterogeneous formats such as CSV files and SQL scripts, which may contain inconsistencies such as missing values, incorrect data types, or formatting issues (especially in date and numeric fields). For example, order-related timestamp fields and pricing attributes may require conversion into standardized formats suitable for analytical processing.

3. ETL Process (SSIS)

The ETL (Extract, Transform, Load) process is implemented using **SQL Server Integration Services (SSIS)**.

- ✓ Extract data from multiple sources
- ✓ Transform data (cleaning, standardization, enrichment)
- ✓ Generate surrogate keys
- ✓ Apply business rules (e.g., SCD Type 2)
- ✓ Load data into the data warehouse

In the context of this **E-commerce Order Processing System**, the SSIS ETL process extracts data from CSV files and SQL sources, transforms it by cleaning, converting data types, and deriving new attributes such as total_amount, and then loads it into dimension and fact tables. Surrogate keys are generated for dimension tables, and lookup transformations are used to map keys before loading the fact table. This process ensures that the data warehouse contains clean, consistent, and structured data for analysis.

4. Data Warehouse Layer

The data warehouse stores integrated and structured data using a **star schema**.

- **Fact Table**
 - ✓ Fact_Sales
- **Dimension Tables**
 - ✓ dim_customer
 - ✓ dim_date
 - ✓ dim_product
 - ✓ dim_seller

In the context of this **E-commerce Order Processing System**, the data warehouse provides a centralized and query-optimized repository for all integrated sales data using a star schema. The central **fact_sales** table captures key transactional measures such as price and freight value, representing each order item.

The surrounding dimension tables provide analytical context across customers, products, sellers, and time. This structure enables efficient analytical queries such as total sales by product category, sales performance by seller, customer purchasing behavior, and monthly sales trends.

5. SSAS / OLAP Cubes

SQL Server Analysis Services (SSAS) is used to create OLAP cubes.

- ✓ Multidimensional analysis
- ✓ Fast query performance
- ✓ Support for hierarchies (Year → Month → Day)
- ✓ Aggregations (e.g., total sales per category)

In the context of this **E-commerce Order Processing System**, SSAS cubes are built on top of the star schema to enable multidimensional analysis of sales data. The time hierarchy (Year → Month → Day) allows users to analyze sales trends over different periods, while product and customer dimensions support analysis by category, location, and purchasing behavior.

6. BI & Reporting Layer

This is the final layer where users interact with the data.

- **Power BI** — primary reporting and visualization tool
- **Interactive Dashboards** — real-time monitoring of sales and order performance.
- **Analytical Reports** — detailed analysis by product, customer, and seller.
- **KPI Monitoring** — tracking key metrics such as total sales and revenue.
- **Trend Analysis** — tracking key metrics such as total sales and revenue (monthly, yearly)

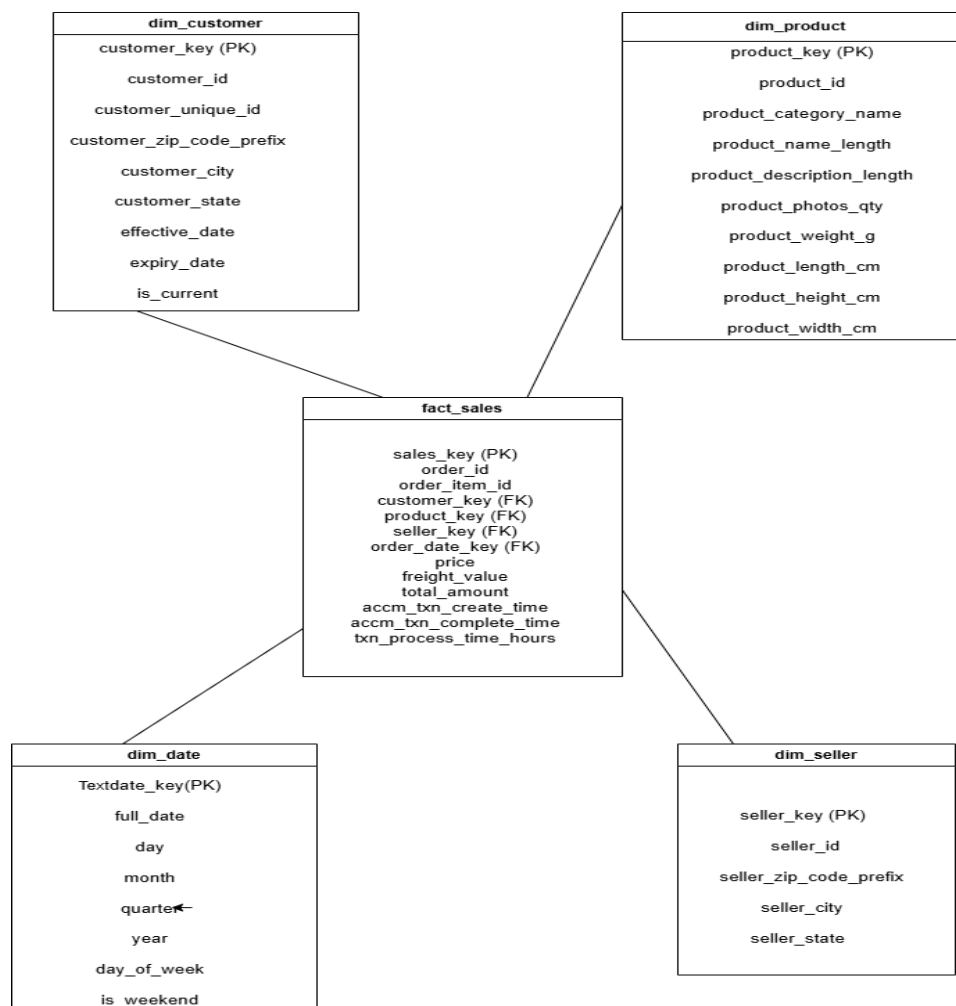
In the context of this **E-commerce Order Processing System**, this layer enables business users to analyze sales performance, monitor customer purchasing behavior, and evaluate product and seller performance. By connecting Power BI to the data warehouse and SSAS cubes, users can explore data through interactive dashboards without needing technical knowledge.

5.Data warehouse design & development

A dimensional model was designed for the **Olist E-commerce marketplace** to support analytical processing, reporting, and future OLAP analysis. Based on the core business process of product sales through online orders, the warehouse was modeled as a **Star Schema**, in which a central fact table is directly connected to several dimension tables. This structure simplifies querying, improves reporting performance, and supports multidimensional analysis across customers, products, sellers, and time.

The grain of the fact table was defined as **one row per order item transaction**. This is the most appropriate grain because the order_items dataset represents the lowest level of transactional detail and contains key measurable values such as price and freight_value required for analysis.

Schema Type :- This schema consists of one central fact table, **FactSales**, directly connected to multiple dimension tables: **DimCustomer**, **DimProduct**, **DimSeller**, and **DimDate**. The dimensions are denormalized and are not split into sub-dimension tables, which confirms that the model is a **Star Schema** rather than a snowflake schema.



FactSales :- The fact_sales table is the central table of the star schema. Each row represents one order item transaction and stores the measurable facts needed for business analysis. The table includes foreign keys to the dimension tables, the natural operational identifiers order_id and order_item_id, and the core additive measures price and freight_value. A computed total_amount column represents the sum of price and freight value. The fact table was further extended with accumulating snapshot columns for transaction creation time, completion time, and total processing hours.

Column	Type	Description
sales_key	INT (PK)	Surrogate primary key – autogenerated
order_id	VARCHAR(50)	Degenerate dimension – natural key from source
order_item_id	INT	Transaction line identifier from source
customer_key	INT (FK)	Links to dim_customer
product_key	INT (FK)	Links to dim_product
seller_key	INT (FK)	Links to dim_seller
order_date_key	INT (FK)	Links to dim_date
price	DECIMAL(10,2)	Product sale amount
freight_value	DECIMAL(10,2)	Shipping amount
total_amount	Computed / DECIMAL	Price + freight value
accm_txn_create_time	DATETIME	Accumulating: time record was created
accm_txn_complete_time	DATETIME	Accumulating: time transaction was completed
txn_process_time_hours	DECIMAL(10,2)	Derived: hours between create and complete

dim_customer (SCD Type 2):- The `dim\_customer` table contains descriptive information about marketplace customers and is derived from the customer master data. This dimension was designed as a Slowly Changing Dimension Type 2 so that historical changes in location-related customer attributes can be preserved. The natural key customer_id is retained, while `customer\_key` is used as the surrogate key for analytical joins.

Column	Type	Description
customer_key	INT (PK)	Surrogate primary key – autogenerated
customer_id	VARCHAR(50)	Natural key from source system
customer_unique_id	VARCHAR(50)	Stable customer identifier
customer_zip_code_prefix	INT	Zip code prefix

customer_city	VARCHAR(100)	Customer city
customer_state	VARCHAR(10)	Customer state
effective_date	DATE	Date this version became active
expiry_date	DATE	Null for active or current record
is_current	CHAR(1)	Y = current record, N = historical record

dim_product:- The 'dim_product' table stores descriptive information about products and is sourced from the cleaned SQL product source. It includes category, text-length indicators, and physical measurement attributes, which support product-based analysis and category summaries.

Column	Type	Description
product_key	INT (PK)	Surrogate primary key
product_id	VARCHAR(50)	Natural key from source
product_category_name	VARCHAR(100)	Product category
product_name_length	INT	Product name length
product_description_length	INT	Product description length
product_photos_qty	INT	Number of product photos
product_weight_g	DECIMAL(10,2)	Weight in grams
product_length_cm	DECIMAL(10,2)	Length in centimetres
product_height_cm	DECIMAL(10,2)	Height in centimetres
product_width_cm	DECIMAL(10,2)	Width in centimetres

dim_seller:- The 'dim_seller' table stores descriptive information about marketplace sellers and supports seller geography analysis. It contains seller location attributes and uses a surrogate key for warehouse joins.

Column	Type	Description
seller_key	INT (PK)	Surrogate primary key
seller_id	VARCHAR(50)	Natural key from source
seller_zip_code_prefix	INT	Zip code prefix
seller_city	VARCHAR(100)	Seller city
seller_state	VARCHAR(10)	Seller state

dim_date: - The 'dim_date' table is a standard date dimension generated programmatically from the available order dates. It supports time hierarchies such as Year > Quarter > Month > Day and allows efficient date-based filtering through an integer 'date_key' in 'YYYYMMDD' format.

Column	Type	Description
date_key	INT (PK)	Surrogate key in YYYYMMDD format
full_date	DATE	Full calendar date
day_number	INT	Day of month
month_number	INT	Month number
month_name	VARCHAR(20)	Month name
quarter_number	INT	Quarter number
year_number	INT	Four-digit year

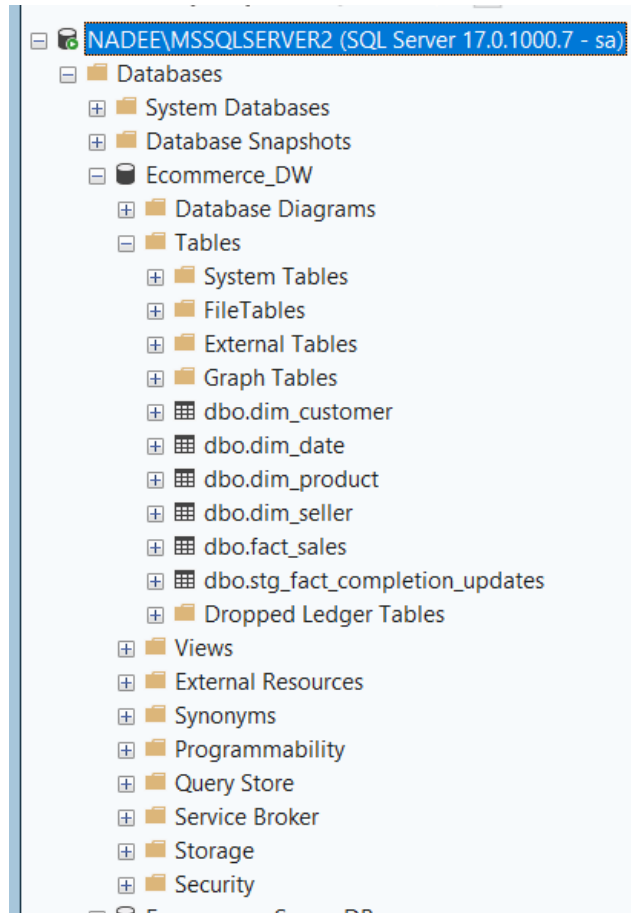
Slowly Changing Dimension (dim_customer) :-

The `dim\_customer` dimension was implemented as an SCD Type 2. This decision was made because customer location attributes such as city, state, or zip code can change over time. When a change is detected, the existing active row is expired and a new version is inserted with a new surrogate key. This allows historical sales analysis to reference the customer version that was valid at the time of the transaction.

Assumptions

- Each fact row represents exactly one order item transaction.
- The `order\_purchase\_timestamp` field is used as the main analytical date for the Date dimension.
- A single order may contain multiple order items, and therefore generate multiple fact rows.
- Price and freight value are treated as additive measures.
- The customer dimension is the only dimension modeled as SCD Type 2 for this assignment.
- Product and seller dimensions are treated as Type 1 overwrite dimensions.

SQL Server Object Explorer

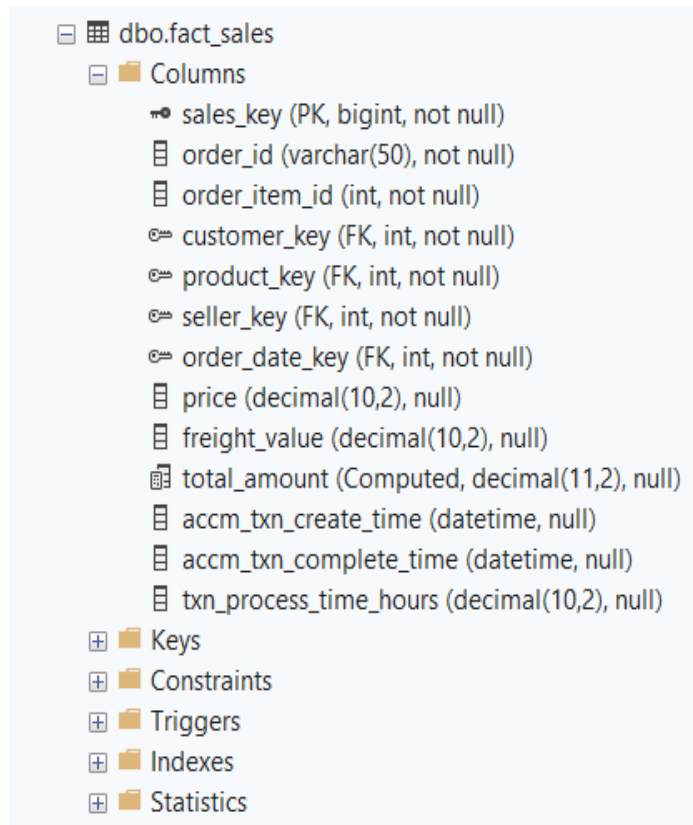


The screenshot above confirms that the data warehouse schema has been successfully implemented in SQL Server Management Studio. The **Ecommerce_DW** database contains one fact table (**fact_sales**) and four dimension tables (**dim_customer**, **dim_date**, **dim_product**, and **dim_seller**), forming a star schema design.

The **fact_sales** table is positioned at the center of the schema and is connected to the dimension tables through surrogate keys. This structure enables efficient querying and supports analytical processing by organizing data into facts and descriptive dimensions.

Table Schema (Columns View)

- **FactSales**



dbo.fact_sales
Columns
sales_key (PK, bigint, not null)
order_id (varchar(50), not null)
order_item_id (int, not null)
customer_key (FK, int, not null)
product_key (FK, int, not null)
seller_key (FK, int, not null)
order_date_key (FK, int, not null)
price (decimal(10,2), null)
freight_value (decimal(10,2), null)
total_amount (Computed, decimal(11,2), null)
accm_txn_create_time (datetime, null)
accm_txn_complete_time (datetime, null)
txn_process_time_hours (decimal(10,2), null)
Keys
Constraints
Triggers
Indexes
Statistics

- The **fact_sales** table stores transactional data related to e-commerce order items. It includes several foreign keys such as **customer_key**, **product_key**, **seller_key**, and **order_date_key**, which establish relationships with the corresponding dimension tables in the star schema.

The table also contains important measures and quantitative attributes, including:

- price
- freight_value
- total_amount

These measures support analytical reporting and enable evaluation of sales performance, revenue analysis, and cost breakdowns.

Additionally, the table is designed as an **accumulating fact table**, incorporating time-based tracking columns such as:

- **accm_txn_create_time** – records when the claim transaction was created
- **accm_txn_complete_time** – records when the claim processing was completed
- **txn_process_time_hours** – calculates the total processing duration

These fields allow tracking of the lifecycle of each claim, enabling performance analysis and monitoring of order processing efficiency in the e-commerce system.

• **DimCustomer (SCD Type 2)**

The **DimCustomer** table is designed as a **Slowly Changing Dimension (Type 2)** to preserve historical changes in customer information. It contains descriptive attributes such as:

- customer_id
- customer_unique_id
- customer_zip_code_prefix
- customer_city
- customer_state

To support historical tracking and versioning, the table includes the following SCD Type 2 columns:

- effective_date – indicates when a record version becomes active
- expiry_date – indicates when a record version expires
- is_current – identifies the current active record

Whenever a customer's details change (such as location updates), a new record is inserted while the previous record is expired. This ensures that historical data is preserved. Whenever a customer's details change (such as location updates), a new record is inserted while the previous record is expired. This ensures that historical data is preserved.

SQL Script Evidence

```
39 -- =====
40 -- DIMENSION: DimCustomer (SCD Type 2)
41 -- Source: customers.sql (SQL Server staging table)
42 -- Tracks historical changes in: occupation, education,
43 -- hobbies, relationship, months_as_customer
44 -- =====
45 CREATE TABLE DimCustomer (
46     customer_key INT NOT NULL IDENTITY(1,1),
47     customer_id NVARCHAR(10) NOT NULL, -- Natural key
48     age TINYINT NOT NULL,
49     insured_sex NVARCHAR(10) NOT NULL,
50     insured_education_level NVARCHAR(30) NOT NULL,
51     insured_occupation NVARCHAR(50) NOT NULL,
52     insured_hobbies NVARCHAR(50) NOT NULL,
53     insured_relationship NVARCHAR(30) NOT NULL,
54     months_as_customer SMALLINT NOT NULL,
55     -- SCD Type 2 tracking columns
56     scd_start_date DATE NOT NULL DEFAULT (GETDATE()) AS DATE),
57     scd_end_date DATE NOT NULL DEFAULT '9999-12-31', -- Sentinel = current record
58     is_current BIT NOT NULL DEFAULT 1,
59     -- SCD Type 2 filter: WHERE scd_end_date = '9999-12-31' OR WHERE is_current = 1
60     CONSTRAINT PK_DimCustomer PRIMARY KEY (customer_key)
61 );
62 GO
63
64 ALTER TABLE dbo.DimCustomer
65 DROP CONSTRAINT DF__DimCustom__scd_s__UCA06362;
66
67 ALTER TABLE dbo.DimCustomer
68 ALTER COLUMN scd_start_date DATETIME NULL;
69
70 ALTER TABLE dbo.DimCustomer
71 ADD CONSTRAINT DF_DimCustomer_scd_start_date
72 DEFAULT (GETDATE()) FOR scd_start_date;
73
74 ALTER TABLE dbo.DimCustomer
75 DROP CONSTRAINT DF__DimCustom__scd_e__4D94879B;
76
77 ALTER TABLE dbo.DimCustomer
78 ALTER COLUMN scd_end_date DATETIME NULL;
79
80 ALTER TABLE dbo.DimCustomer
81 ADD CONSTRAINT DF_DimCustomer_scd_end_date
82 DEFAULT ('9999-12-31') FOR scd_end_date;
83
84 -- Index on natural key for ETL lookups
85 CREATE INDEX IX_DimCustomer_NaturalKey
86 ON DimCustomer (customer_id, is_current);
87 GO
88
89 -- =====
90 -- FACT TABLE: FactClaims
91 -- Grain: One row per insurance claim
92 -- Source: All three sources combined via ETL
93 -- Includes accumulating snapshot columns (Task 6)
94 -- =====
95 CREATE TABLE FactClaims (
96     claim_key INT NOT NULL IDENTITY(1,1),
97     -- Natural key (degenerate dimension)
98     claim_id NVARCHAR(15) NOT NULL,
99     -- Foreign keys to dimension tables
100     customer_key INT NOT NULL,
101     vehicle_key INT NOT NULL,
102     policy_key INT NOT NULL,
103     incident_key INT NOT NULL,
104     date_key INT NOT NULL, -- Links to DimDate (incident date)
105     -- Measures (additive facts)
106     injury_claim INT NOT NULL DEFAULT 0,
107     property_claim INT NOT NULL DEFAULT 0,
108     vehicle_claim INT NOT NULL DEFAULT 0,
109     total_claim_amount INT NOT NULL DEFAULT 0,
110     -- Semi-additive fact (flag)
111     fraud_reported NCHAR(1) NOT NULL, -- 'Y' or 'N'
112     CONSTRAINT CK_FactClaims_FraudReported CHECK (fraud_reported IN ('Y', 'N')),
113     -- Accumulating snapshot columns (Task 6)
114     accm_txn_create_time DATETIME NOT NULL DEFAULT GETDATE(),
115     accm_txn_complete_time DATETIME NULL,
116     txn_process_time_hours DECIMAL(10,2) NULL,
117     CONSTRAINT PK_FactClaims PRIMARY KEY (claim_key),
118     CONSTRAINT FK_Fact_Customer FOREIGN KEY (customer_key) REFERENCES DimCustomer (customer_key),
119     CONSTRAINT FK_Fact_Vehicle FOREIGN KEY (vehicle_key) REFERENCES DimVehicle (vehicle_key),
120     CONSTRAINT FK_Fact_Policy FOREIGN KEY (policy_key) REFERENCES DimPolicy (policy_key),
121     CONSTRAINT FK_Fact_Incident FOREIGN KEY (incident_key) REFERENCES DimIncident (incident_key),
122     CONSTRAINT FK_Fact_Date FOREIGN KEY (date_key) REFERENCES DimDate (date_key)
123 );
124 GO
125
126 CREATE UNIQUE INDEX IX_FactClaims_NaturalKey
127 ON FactClaims (claim_id);
128 GO
129
130 -- Index to support accumulating fact ETL lookup (Task 6)
131 CREATE INDEX IX_FactClaims_AccumLookup
132 ON FactClaims (claim_id, accm_txn_complete_time);
133 GO
```

6.ETL Development

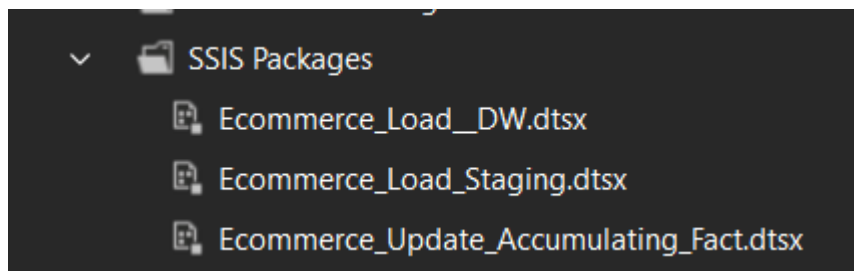
To populate the data warehouse with clean, integrated, and analysis-ready data, several ETL packages were developed using **SQL Server Integration Services (SSIS)**. The ETL process was divided into three main stages: **data extraction into staging, transformation and loading into the data warehouse, and updating the accumulating fact table.**

This modular design improves maintainability, simplifies debugging, and ensures a clear separation of ETL responsibilities.

SSIS Packages Overview

The following SSIS packages were created to support the ETL workflow.

Package Name	Purpose
Ecommerce_Load_Staging.dtsx	Extracts raw data from source structures and loads it into staging tables
Ecommerce_Load_DW.dtsx	Transforms staged data and loads it into dimensions and fact table
Ecommerce_Update_Accumulating_Fact.dtsx	Updates accumulating fact columns such as completion time and process duration



Connection Managers

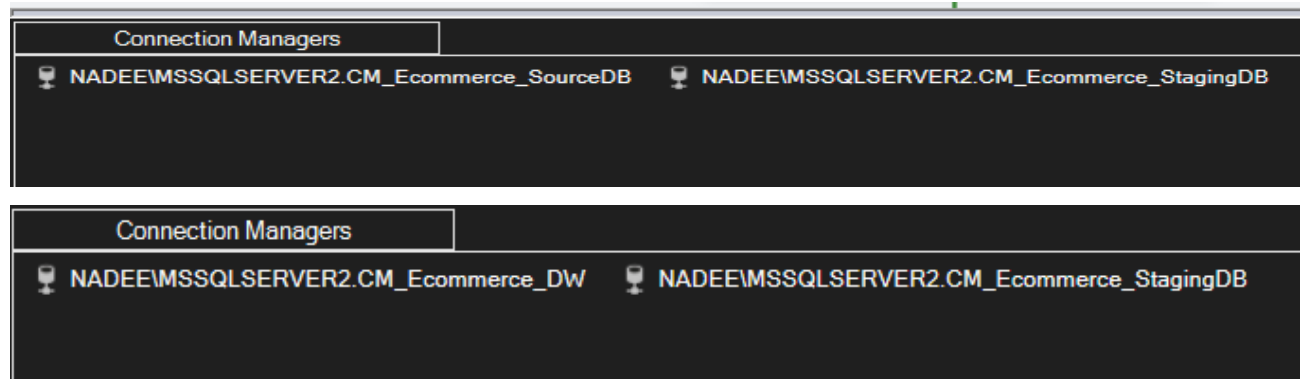
All ETL packages use multiple connection managers to integrate data from different sources and destinations.

➤ OLE DB Connection Managers

- **CM_Ecommerce_SourceDB** - connects to the source operational database ('Ecommerce_SourceDB') containing raw imported source tables
- **CM_Ecommerce_StagingDB** - connects to the staging database ('Ecommerce_StagingDB') used for intermediate preparation.
- **CM_Ecommerce_DW** - connects to the final data warehouse ('Ecommerce_DW').

➤ Flat File Connection Managers

- **Flat File Connection Manager(s)** - used to read external data files containing insurance-related information such as **claims data and vehicle details**, which are used during the ETL process.



Package 1: Ecommerce_Load_Staging.dtsx

This package extracts raw data from source systems and loads it into staging tables. The control flow consists of the following sequential tasks:

1. **Extract Customer Data to Staging**
2. **Extract Order Data to Staging**
3. **Extract Order Items Data to Staging**
4. **Extract Sellers Data to Staging**
5. **Extract Product Data to Staging**

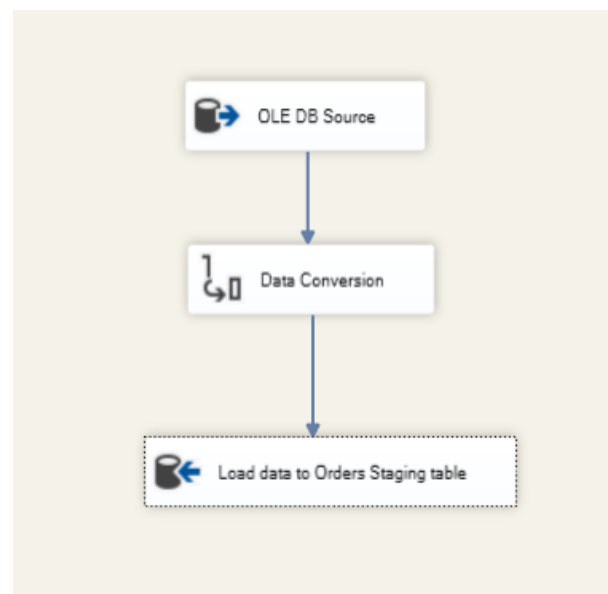
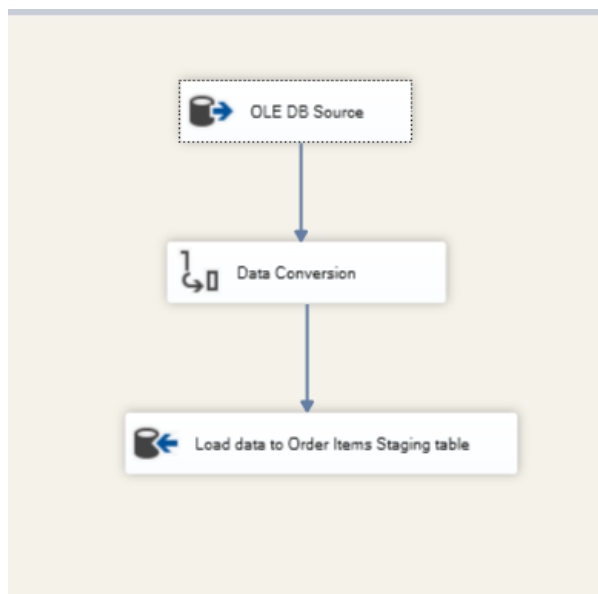
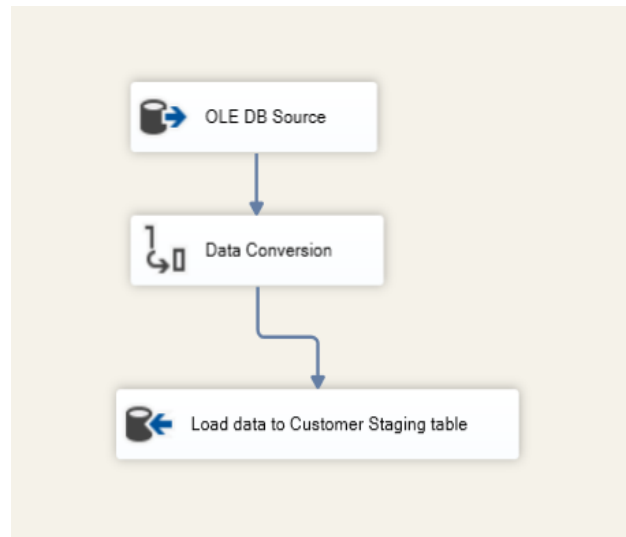
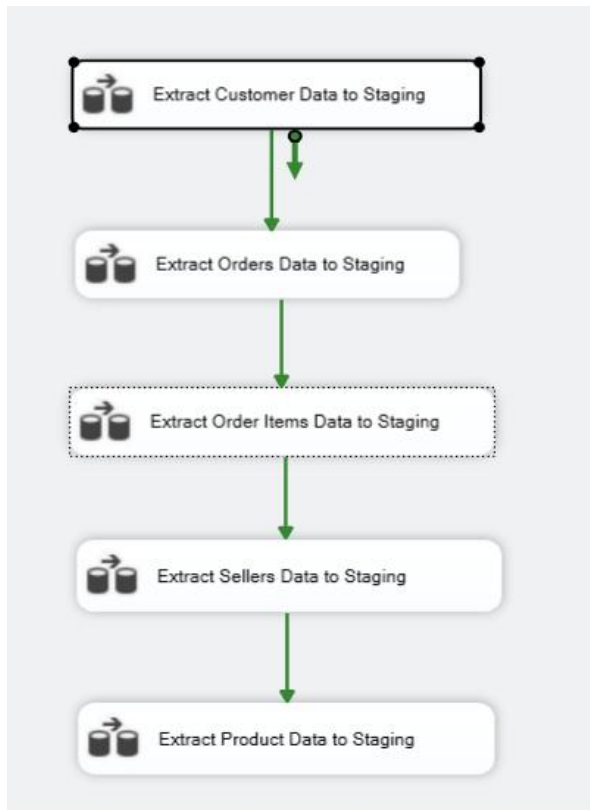
Each extraction task is implemented using a **Data Flow Task**, where appropriate SSIS components are used depending on the data source type.

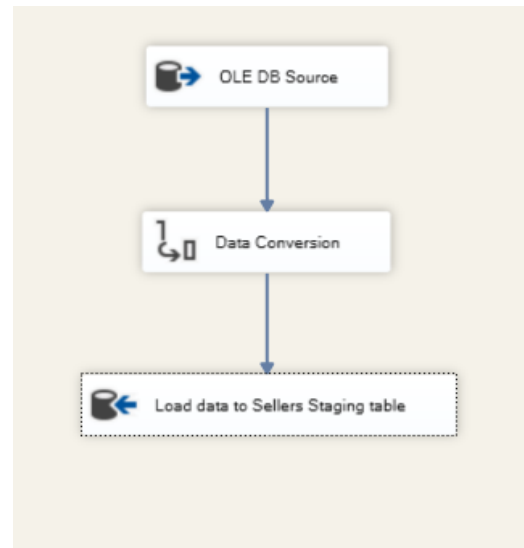
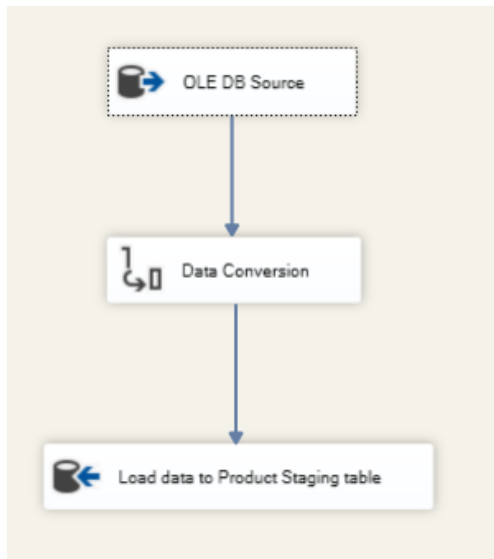
Customer, orders, order items, and seller data are extracted from structured sources (CSV/Excel or database) and loaded into corresponding staging tables (**stg_customers**, **stg_orders**, **stg_order_items**, **stg_sellers**) using **Source** → **OLE DB Destination** data flows.

Product data is loaded from a SQL script into the source database and then transferred into the staging table (**stg_products**) using an **OLE DB Source to OLE DB Destination** pipeline.

All tasks are executed sequentially to ensure proper data loading order and to maintain data consistency across staging tables.

The staging layer acts as a temporary storage area where data is prepared for further transformation before being loaded into the data warehouse in the next package (**Ecommerce_Load_DW.dtsx**).





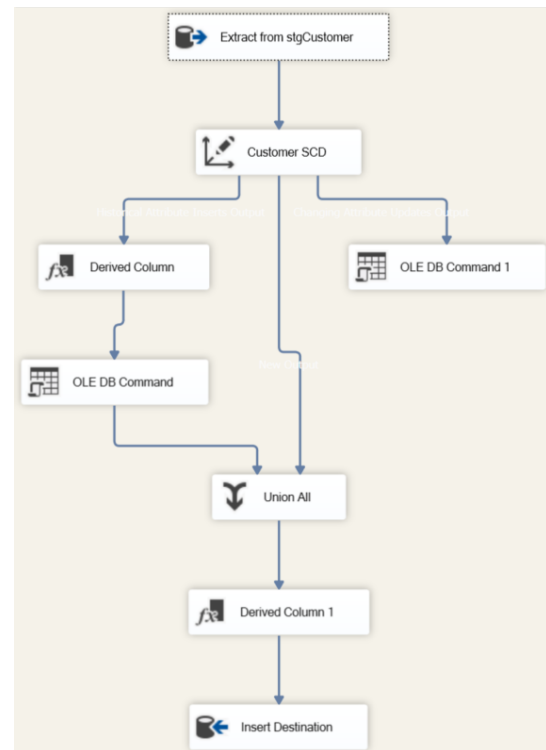
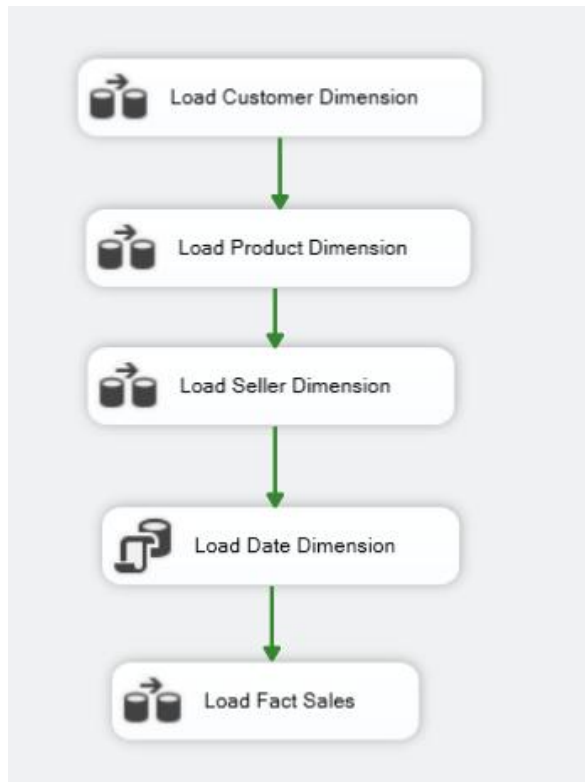
Package 2: Ecommerce_Load_DW.dtsx

This package handles the transformation and loading of clean data from the staging tables into the dimension and fact tables of the data warehouse. The tasks are executed in the following order:

1. Load Customer Dimension
2. Load Product Dimension
3. Load Seller Dimension
4. Load Date Dimension
5. Load Fact Sales

Within the data flows, **Lookup transformations** are used to resolve surrogate keys, while **Derived Column transformations** are used to create new attributes and calculate required fields.

Transform and Load dim_customer



This data flow extracts records from the **stg_customers** table and processes them using a **Slowly Changing Dimension (SCD Type 2)** transformation.

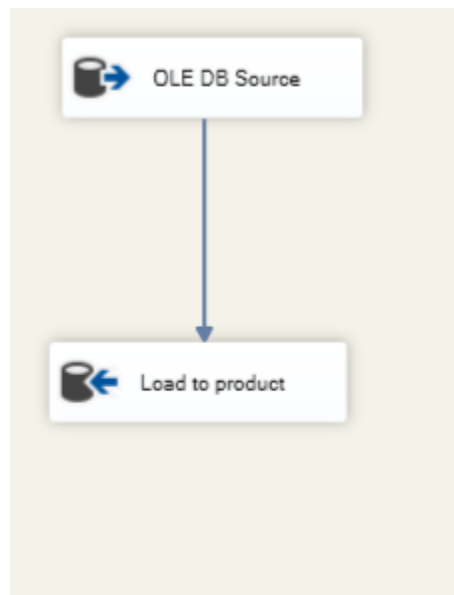
The SCD component separates records into:

- **New records**, which are inserted into **dim_customer**
- **Changing records**, where existing records are updated by setting the **expiry_date**, and a new version of the record is inserted

Derived Column transformations are used to assign values such as **effective_date**, **expiry_date**, and **is_current** before loading the data. A Union All transformation can be used to combine outputs if required before inserting into the destination table.

This design ensures that historical changes in customer data (such as location updates) are preserved over time, enabling accurate time-based analysis in the e-commerce system.

Transform and Load dim_product



This data flow extracts product data from the **stg_products** table and loads it into the **dim_product** table.

During the transformation phase, necessary data cleaning and standardization are applied, including handling NULL values and ensuring correct data types for product attributes. Derived Column transformations may be used to format or prepare attributes before loading.

The processed data is then inserted into the **dim_product** table using an OLE DB Destination. This dimension stores descriptive product information such as category and physical attributes, which are essential for product-level analysis in the data warehouse.

Transform and Load dim_seller

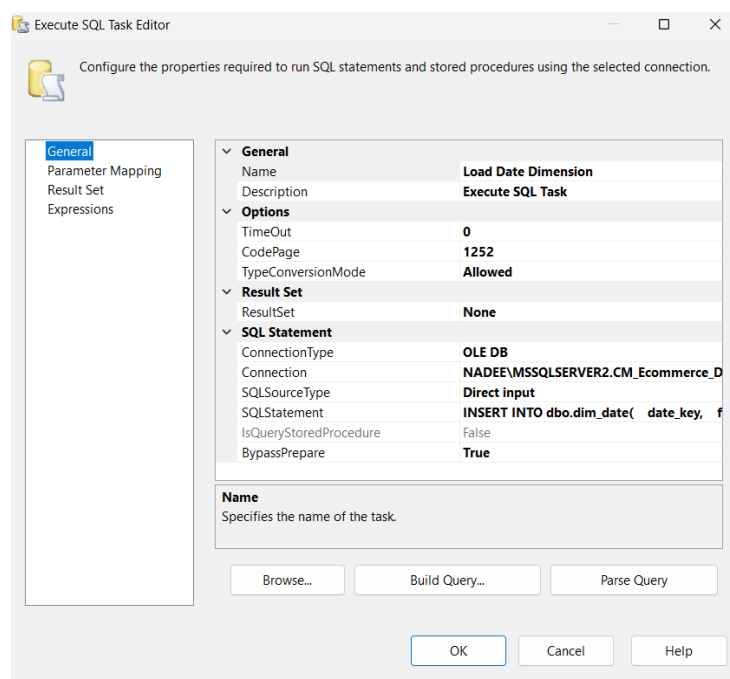


This data flow extracts seller data from the **stg_sellers** table and loads it into the **dim_seller** table.

Data transformations are applied to ensure consistency and correctness, including standardizing location-related attributes such as city and state. Any necessary data type conversions or cleansing operations are performed before loading.

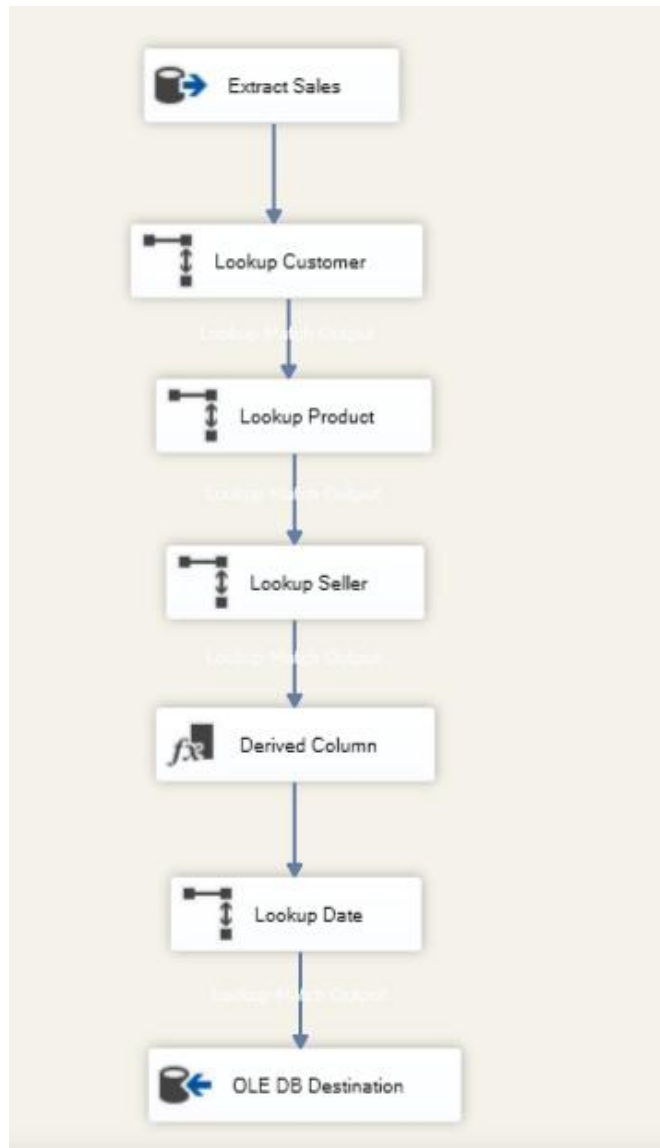
The transformed data is then inserted into the **dim_seller** table using an OLE DB Destination. This dimension provides contextual information about sellers, enabling analysis of sales performance by seller and location in the e-commerce system.

Transform and Load dim_date



The **dim_date** table is populated by generating date-related records required for time-based analysis. Transformations are applied to derive attributes such as day, month, quarter, and year. This table is used to support time hierarchies and enables analysis of sales trends over different time periods.

Transform and Load FactSales



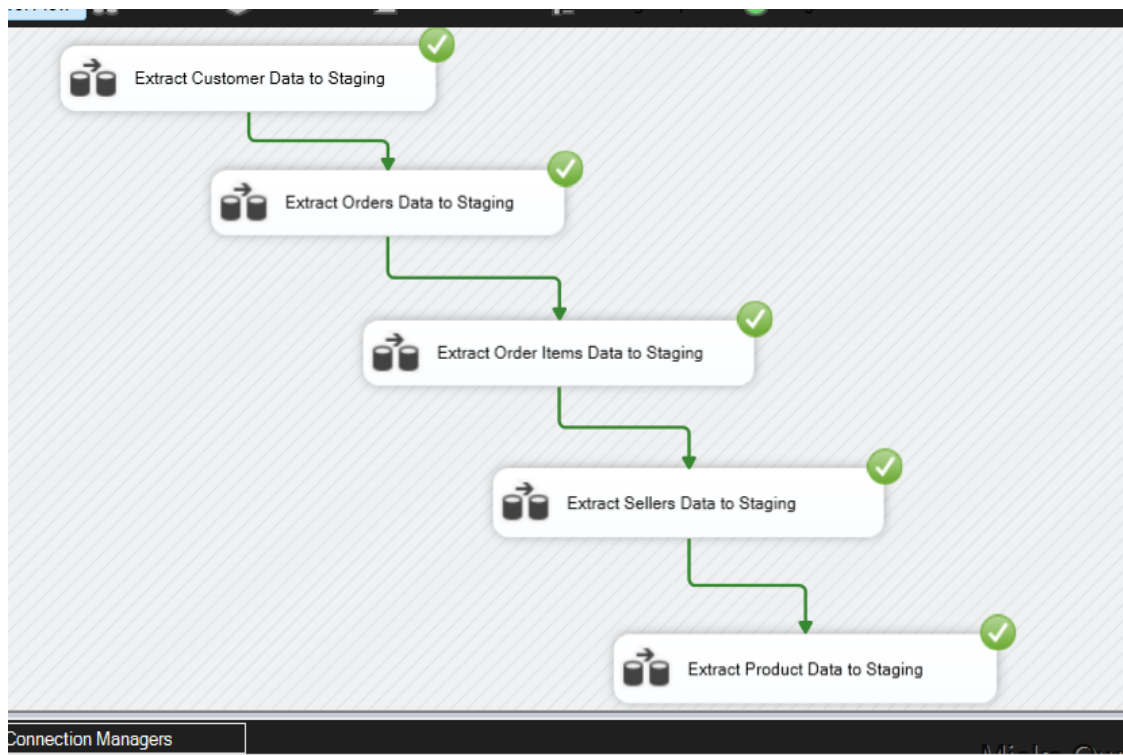
This data flow extracts transactional data from staging tables (**stg_orders** and **stg_order_items**) and transforms it into the **fact_sales** table. Lookup transformations are used to map business keys (customer_id, product_id, seller_id, and order date) to surrogate keys from dimension tables.

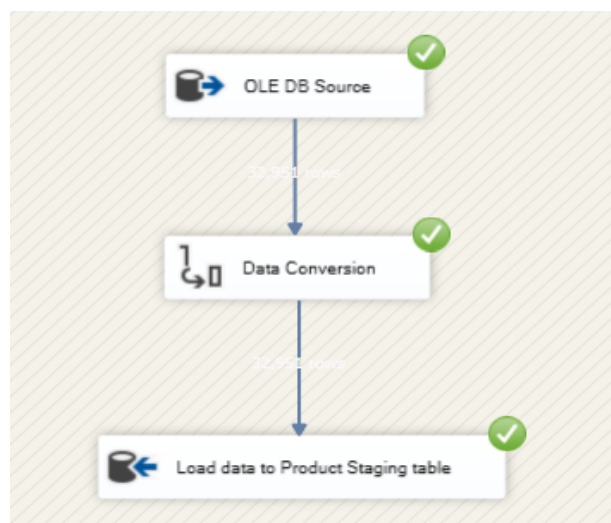
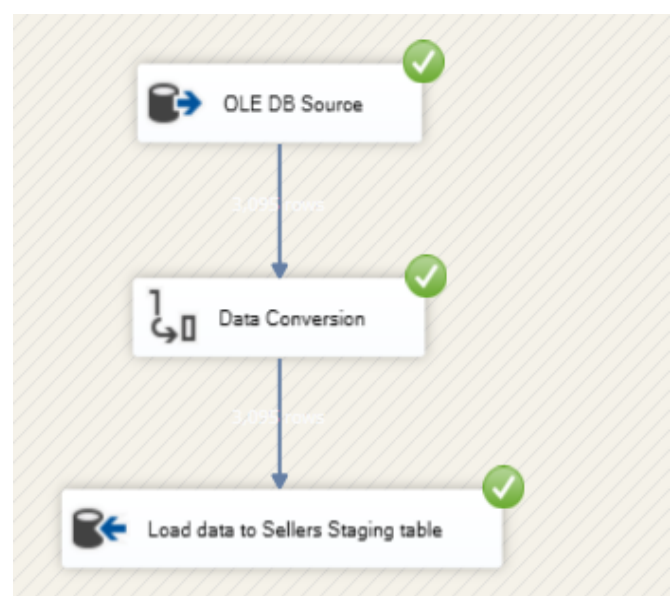
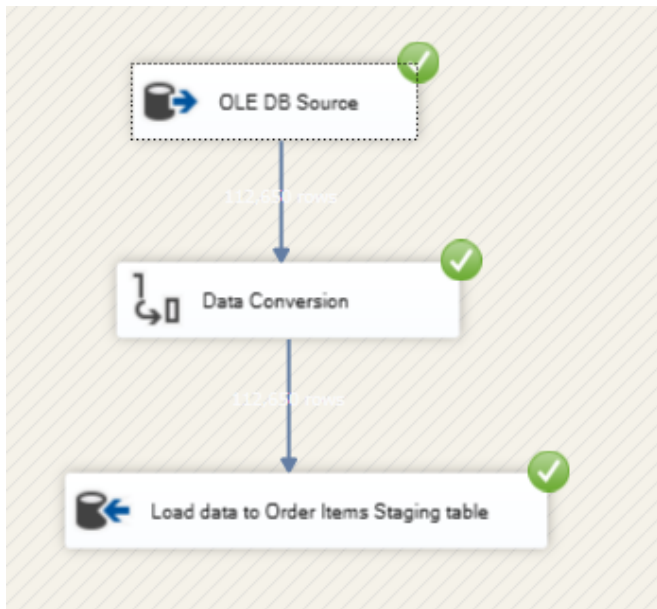
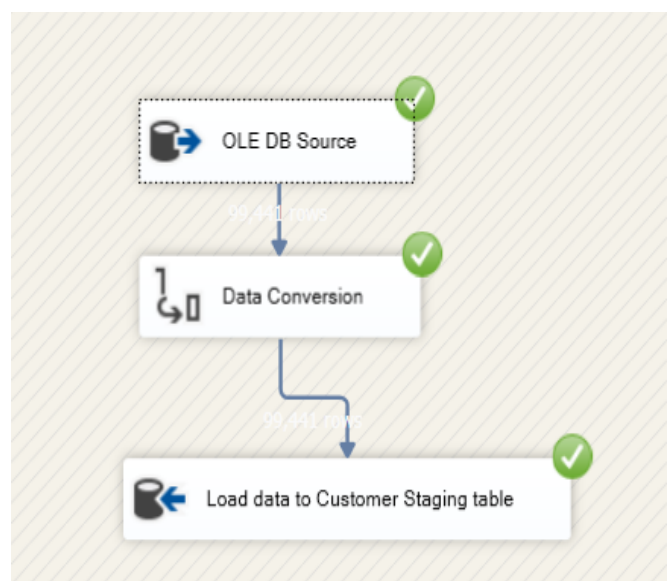
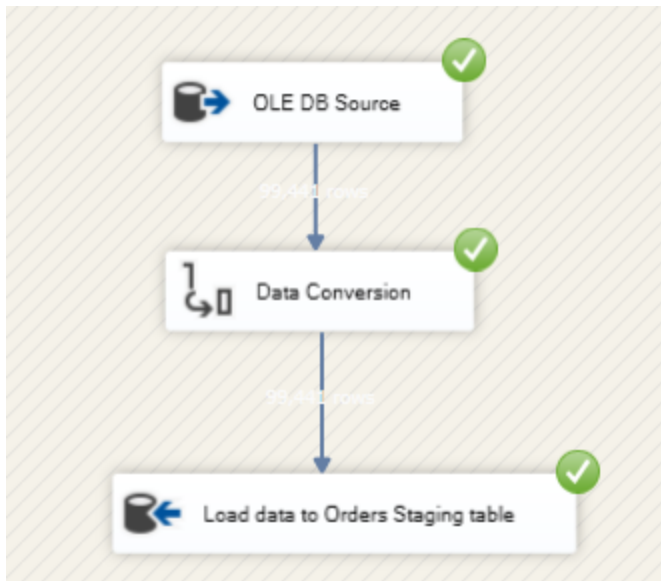
Derived Column transformations are used to calculate measures such as **total_amount (price + freight_value)**. The final transformed data is then loaded into the fact table using an OLE DB Destination. This table captures detailed sales transactions and supports analytical queries across multiple dimensions.

After resolving surrogate keys, a **Derived Column** transformation is used to set the `accm_txn_create_time` field. A **Data Conversion** transformation ensures correct data types before inserting the final records into the FactClaims table.

This process ensures that all fact records are properly linked to dimension tables, enabling efficient analytical queries.

Evidence of Row Counts: Data Load into Staging Tables





```
1 select 'stg_customers' AS TableName, COUNT(*) AS Row_Count FROM dbo.stg_customers
2 UNION ALL
3 select 'stg_order_items' AS TableName, COUNT(*) AS Row_Count FROM dbo.stg_order_items
4 UNION ALL
5 select 'stg_orders' AS TableName, COUNT(*) AS Row_Count FROM dbo.stg_orders
6 UNION ALL
7 select 'stg_products' AS TableName, COUNT(*) AS Row_Count FROM dbo.stg_products
8 UNION ALL
9 select 'stg_sellers' AS TableName, COUNT(*) AS Row_Count FROM dbo.stg_sellers;
10
```

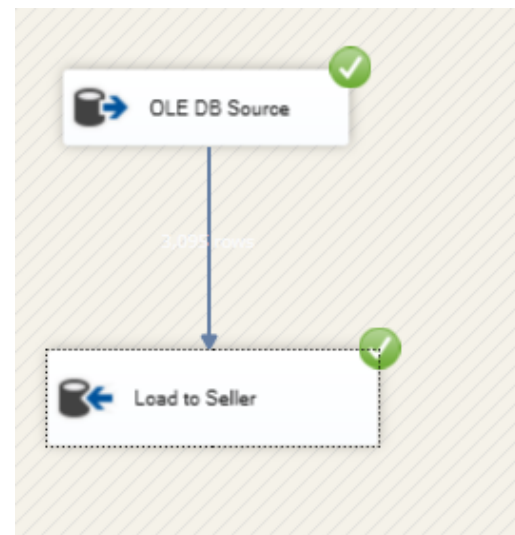
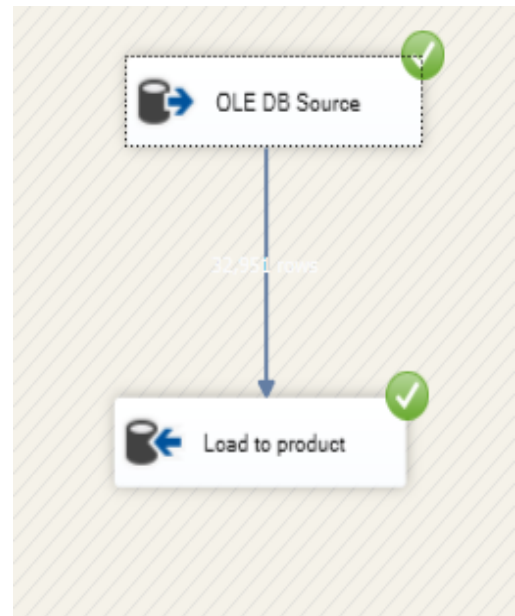
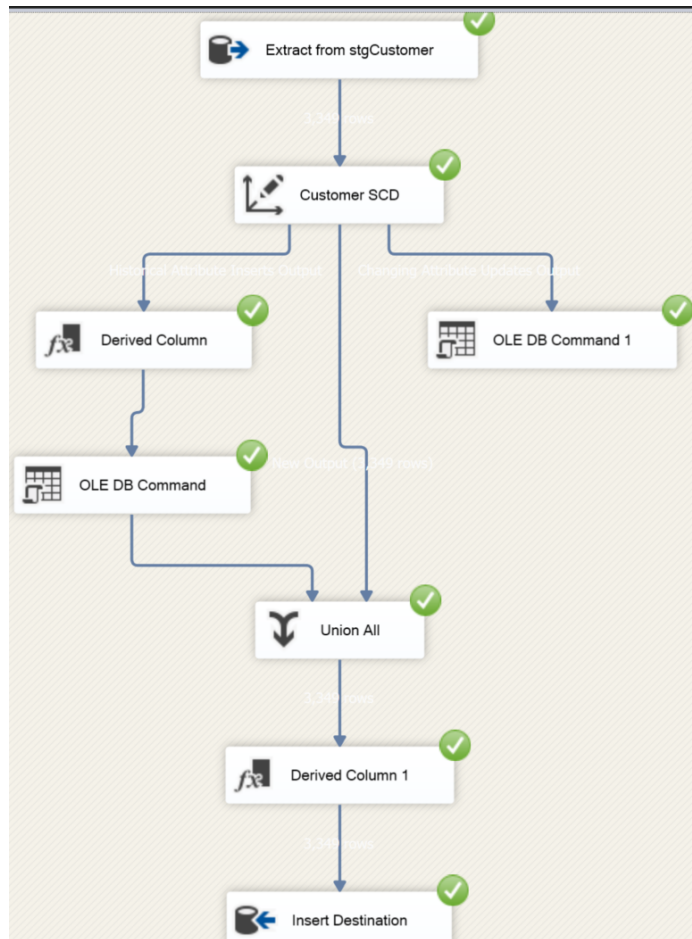
74 % ✓ No issues found

Results Messages

	TableName	Row_Count
1	stg_customers	99441
2	stg_order_items	112650
3	stg_orders	99441
4	stg_products	32951
5	stg_sellers	3095

Evidence of Row Counts: Data Load into Data Warehouse Tables



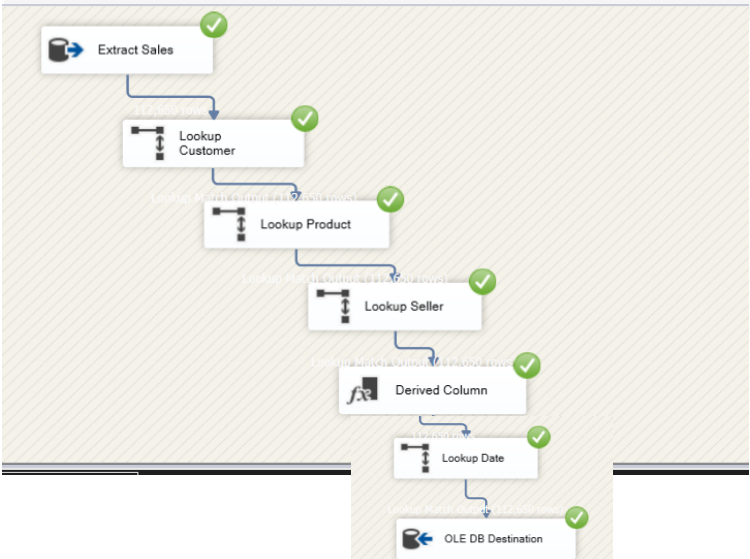


```
1 SELECT 'dim_date' AS TableName, COUNT(*) AS [RowCount] FROM dbo.dim_date
2 UNION ALL
3 SELECT 'dim_customer', COUNT(*) FROM dbo.dim_customer
4 UNION ALL
5 SELECT 'dim_product', COUNT(*) FROM dbo.dim_product
6 UNION ALL
7 SELECT 'dim_seller', COUNT(*) FROM dbo.dim_seller
8 UNION ALL
9 SELECT 'fact_sales', COUNT(*) FROM dbo.fact_sales;
```

74 % No issues found

Results Messages

	TableName	RowCount
1	dim_date	634
2	dim_customer	701499
3	dim_product	230657
4	dim_seller	15475
5	fact_sales	225300



7.ETL development – Accumulating fact tables

Package 3: Insurance_Update_AccumulatingFact.dtsx

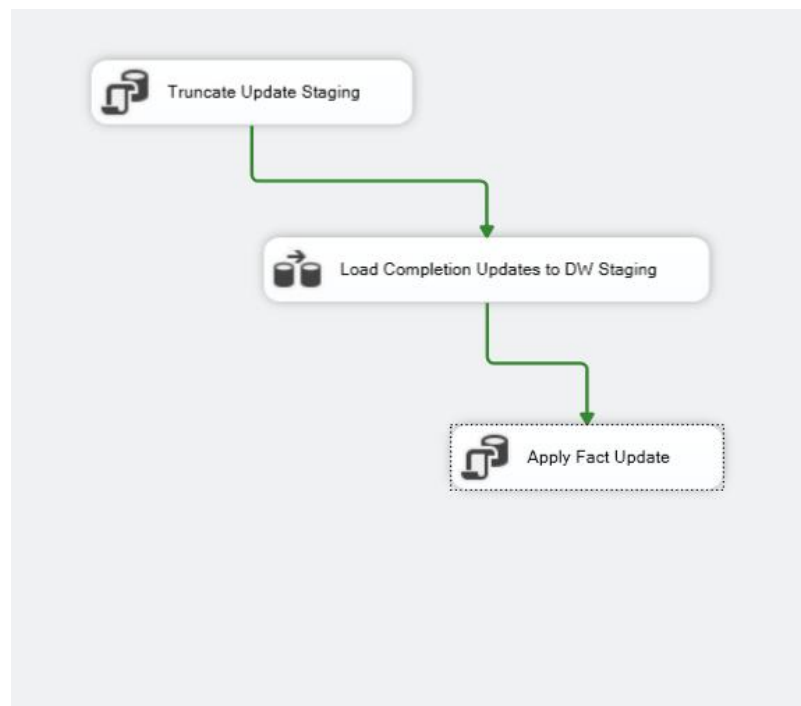
This package demonstrates the implementation of **accumulating snapshot logic** by updating the existing **fact_sales** table using order completion data. The purpose of this package is to populate the **accm_txn_complete_time** field and calculate the **txn_process_time_hours** metric after the initial fact records have already been loaded.

The control flow consists of the following steps:

- **Truncate Update Staging** – Clear the temporary staging table used for completion updates
- **Load Completion Updates to DW Staging** – Load order completion records into the staging table
- **Apply Fact Update** – Update the **accm_txn_complete_time** field in the fact table and calculate the processing duration.

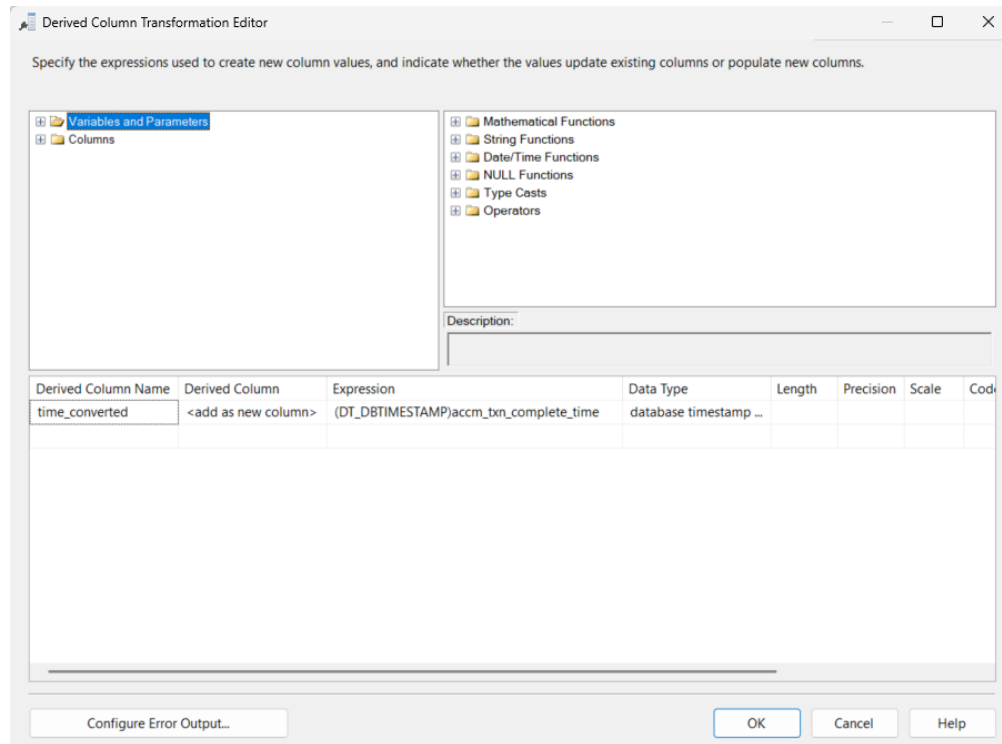
This design allows the fact table to behave as an **accumulating snapshot fact table**, where transaction completion details are added later while preserving the original creation timestamp.

The package supports tracking of order lifecycle performance by storing both the transaction creation time and the completion time. The difference between these two timestamps is calculated and stored in **txn_process_time_hours**, enabling analysis of order processing efficiency in the e-commerce system.



Data Flow – Read Completion Update Data

The update data flow reads `order\_id`, `order\_item\_id`, and `accm\_txn\_complete\_time` from the source update table and inserts them into the warehouse-side staging table. These columns form the business key plus completion timestamp needed for the accumulating fact update.



Fact Table Update – Completion Time and Process hours.

The final update is implemented using a set-based SQL statement that joins `fact\_sales` with `stg\_fact\_completion\_updates` on `order\_id` and `order\_item\_id`. The update sets `accm\_txn\_complete\_time` directly and calculates `txn\_process\_time\_hours` using `DATEDIFF(MINUTE, accm\_txn\_create\_time, accm\_txn\_complete\_time) / 60.0`. This enables the warehouse to track transaction processing duration in hours.

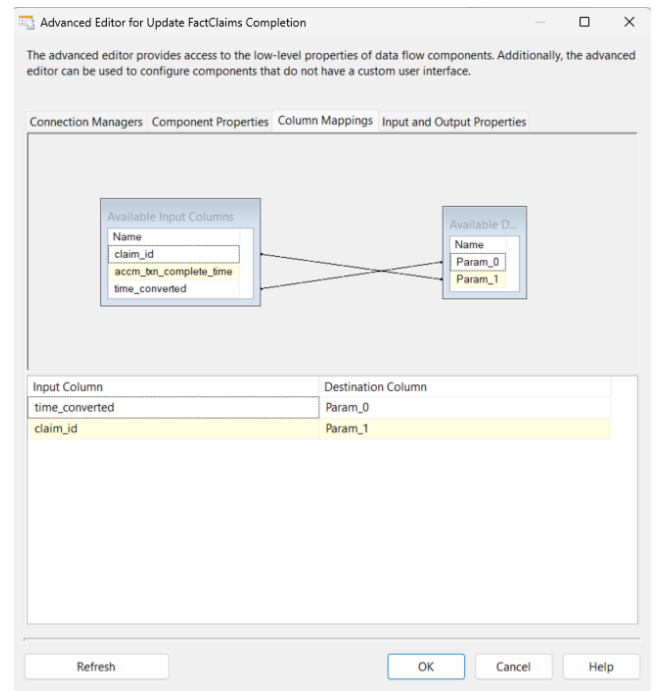
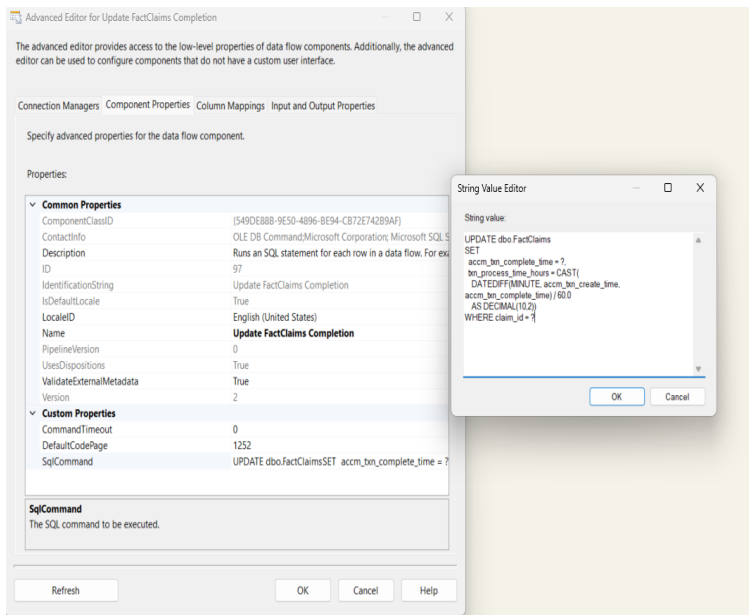
The update statement sets:

- **accm_txn_complete_time** = the converted completion timestamp
- **txn_process_time_hours** is also calculated in the SQL command in your current implementation

The mapping shows that:

- **time_converted** is passed as **Param_0**
- **claim_id** is passed as **Param_1**

This ensures that the correct fact row is updated using its natural key.

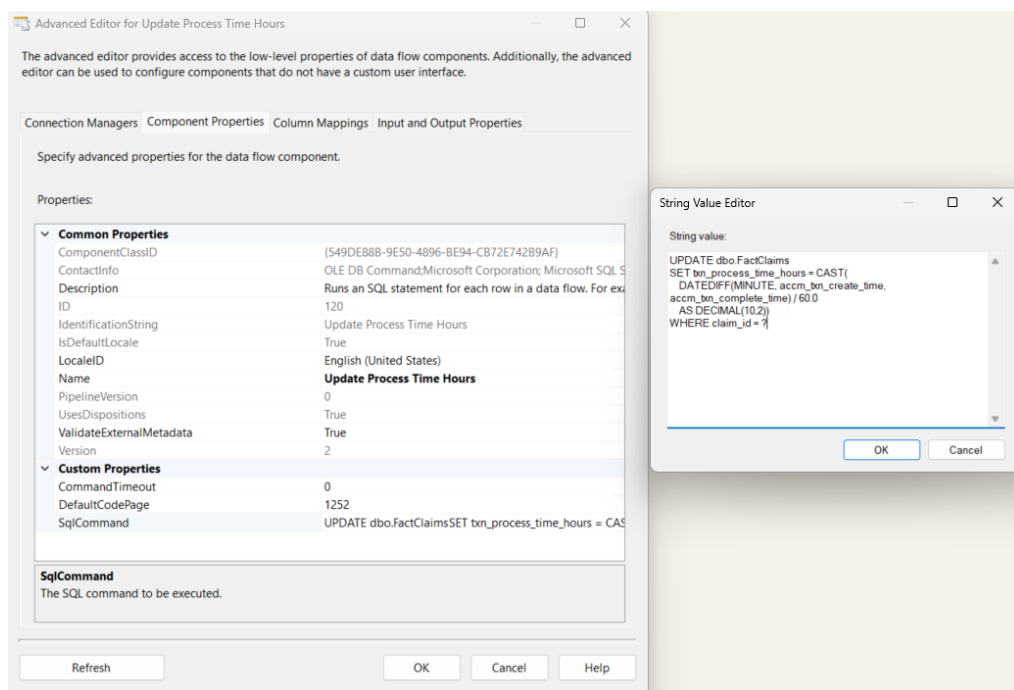


Fact Table Update – Process Duration in Hours

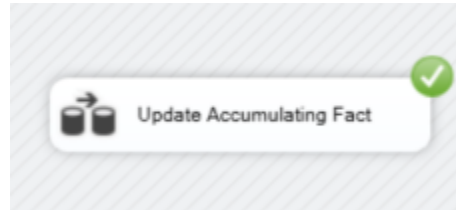
A second update step is used to calculate the total transaction processing duration in hours. This is done by comparing the original **accm_txn_create_time** with the newly updated **accm_txn_complete_time**.

The SQL logic uses **DATEDIFF(MINUTE, accm_txn_create_time, accm_txn_complete_time) / 60.0** and casts the result into **DECIMAL(10,2)** before storing it in **txn_process_time_hours**.

This derived metric supports analysis of claim processing efficiency and completion duration.



Evidence of Accumulating Fact Table Update



```
--
12
13
14      SELECT
15      order_id,
16      order_item_id,
17      accm_txn_create_time,
18      accm_txn_complete_time,
19      txn_process_time_hours
20      FROM dbo.fact_sales
21      WHERE order_id IN (
22      '00042b26cf59d7ce69dfabb4e55b4fd9',
23      '00048cc3ae777c65dbb7d2a0634bc1ea',
24      '000576e39319847cbb9d288c5617fa6'
25      )
26      ORDER BY order_id;
```

74 %  No issues found

Results Messages

	order_id	order_item_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	00042b26cf59d7ce69dfabb4e55b4fd9	1	2026-03-28 11:42:17.377	2026-03-28 22:43:48.060	11.02
2	00042b26cf59d7ce69dfabb4e55b4fd9	1	2026-03-30 02:49:59.133	NULL	NULL
3	00048cc3ae777c65dbb7d2a0634bc1ea	1	2026-03-30 02:49:59.133	NULL	NULL
4	00048cc3ae777c65dbb7d2a0634bc1ea	1	2026-03-28 11:42:17.377	2026-03-29 03:43:48.060	16.02

8. Conclusion

In this assignment, a complete data warehouse solution was designed and implemented for an **E-commerce Order Processing System**. To simulate a real-world business environment, data from multiple heterogeneous sources such as SQL scripts and CSV/Excel files were integrated into a centralized system.

SQL Server Integration Services (SSIS) was used to develop the ETL process, enabling efficient extraction, transformation, and loading of data into a structured format. The data warehouse was designed using a **star schema**, with a central **fact_sales** table and multiple dimension tables, allowing efficient querying and analytical processing.

Key data warehousing techniques such as **Slowly Changing Dimensions (Type 2)**, **surrogate key generation**, and **accumulating snapshot logic** were successfully implemented. The accumulating fact table design enabled tracking of order lifecycle events, including transaction creation, completion time, and processing duration.

Although advanced reporting tools were not fully implemented in this assignment, the developed data warehouse is fully prepared for integration with business intelligence tools such as **Power BI and SSAS**.

Overall, this project provided valuable practical experience in dimensional modeling, ETL development, and data integration within a business intelligence environment. It also highlighted the importance of data warehousing in supporting data-driven decision-making in e-commerce operations.